

МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ

**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»**

Институт Компьютерных наук и технологий

Направление 02.03.01 Математика и компьютерные науки

ОТЧЕТ ПО КУРСОВОЙ РАБОТЕ:

Реализация класса SmartBot для игры "UNO

Обучающийся: _____

Санько В. В.

Руководитель: _____

Сеннов В. Н.

« _____ » _____ 20__ г.

Санкт-Петербург, 2023

Содержание

1	Постановка задач	3
2	Описание алгоритмов и реализации	4
2.1	Математическое описание алгоритмов классов игроков	4
2.1.1	UnoBot	4
2.1.2	SmartBot	4
2.2	Реализация алгоритмов	4
2.2.1	UnoBot	4
2.2.2	SmartBot	5
3	Результаты	8
	Выводы	9
	Приложение А. Исходный код класса SmartBot	10
A.1.	Исходный лист	10
A.2.	SmartBot.cpp	12
	Приложение Б. Исходный код класса UnoBot	20
Б.1.	UnoBot.h	20
Б.2.	UnoBot.cpp	22

1 Постановка задач

Цель: Научиться взаимодействовать с готовым кодом. В данном случае необходимо создать класс игрока, который будет способен участвовать в игре, написанной преподавателем, при этом соблюдая все правила этой игры. Главное условие которое обязан выполнять класс игрока - это не оказаться дисквалифицированным. Здесь предоставлена ссылка на исходный [код](#) игры.

Задачи, которые необходимо выполнить.

1. Реализация метода **receiveCards**(получить карты в колоду).
2. Реализация метода **changeColor**(поменять цвет).
3. Реализация метода **playCard**(сделать ход).
4. Реализация метода **drawAdditionalCard**(взять дополнительную карту).

2 Описание алгоритмов и реализации

2.1 Математическое описание алгоритмов классов игроков

2.1.1 UnoBot

Реализация класса **UnoBot** не включает в себя ничего необычного. **UnoBot** ходит случайной картой, которая удовлетворяет условиям игры. Карта может подходить по цвету или по значению, однако **UnoBot** может ходить картой смены цвета, если других доступных ходов нет. Цвет меняется на самый редкий из всех, что есть на руках. Это сделано для того, чтобы дать преимущество сильному боту и ослабить **UnoBot**. Если нет карт удовлетворяющих условиям игры **UnoBot** берет одну карту. При условии, что набранная карта удовлетворяет условиям, **UnoBot** походит набранной картой, в противном случае - заберет карту себе.

2.1.2 SmartBot

Класс **SmartBot** немного сложнее, чем **UnoBot**. Он получает карты и определяет их цвет и значение. Все методы описанные в **UnoBot** есть и в **SmartBot**, но помимо них есть также метод, который запоминает все карты попавшие в сброс и метод, сравнивающий значения карт в сбросе с теми картами, что попадают в руки. Карты с наиболее частым цветом и значением будут попадать в сброс быстрее. Это сделано для того, чтобы избавиться в первую очередь от карт с меньшим шансом выпадения, поскольку шанс того, что последняя карта окажется подходящей по цвету и значению будет значительно меньше, по сравнению с теми картами, которые у бота на руке и которые остались в колоде. В этой логике предусмотрено то, что после окончания карт в колоде, сброс перемешивается и помещается в колоду заново. Последнее отличие заключается в выборе цвета. Для **SmartBot** этот метод реализован иначе. Он находит самый частый цвет, для того чтобы облегчить себе игру.

2.2 Реализация алгоритмов

2.2.1 UnoBot

Рассмотрим все методы которые есть в этом классе.

1. **string name() const** - возвращает имя игрока.

2. **void receiveCards(const vector<const Card*> cards)** - метод который отвечает за получение карт на руку.

В данном методе класс игрока получает карты в руку во время игры, копирует их в поле класса **Hand**, в то же время считая количество цветов и карточных действий в поле класса **clrs**. Для того, чтобы в новой партии у него на руках не оставались карты с предыдущей, используем метод **clear()** к методу класса **Hand**. Но используется он в перегруженном методе **void**

handleSetStarted (int gameNumber) который оповещает всех игроков о старте новой партии.

3. **const Card* playCard()** - этот метод позволяет ходить картой с руки.

Карты перебираются по порядку пока не найдется подходящая карта. Если первой картой удовлетворяющей условию стала "Дикая карта" , тогда перебор пройдет дальше в поиске другой карты. Если карт подходящих по цвету и значению не нашлось, играть будет "Дикая карта" , если нет ни одной подходящей карты, бот возьмет карту из колоды.

4. **bool isMatchingCard(const Card* card) const** - функция проверяет подходит ли карта на руках для этого хода.

Функция проверяет поочередно карты на руке по цвету и значению. Если карта подходит по цвету или по значению, то функция выдаст **true**, а также результат будет **true**, если набранная карта является картой типа "Дикая карта" , в остальных случаях функция выдаст **false**.

5. **void playMatchingCard(const Card* card)** - этот метод нужен для того, чтобы сыграть картой.

Если результат функции **bool isMatchingCard(const Card* card) const true**, значит карта с руки подходящая для игры в этом раунде, а значит она будет сыграна. Функция также позволяет играть только что набранными картами из колоды.

6. **bool drawAdditionalCard(const Card* additionalCard)** - Функция, определяющая подходит ли набранная карта из колоды для того, чтобы сыграть ее в этом же раунде.

Набранная карта сравнивается по цвету и по значению с последней картой на столе, а также принадлежит ли набранная карта к типу "Дикая карта" .Если карта соответствует по цвету, или по значению, или принадлежит к типу "Дикая карта" , Функция выводит **true**, если же нет **false**.

7. **CardColor changeColor()** - идея метода в том, чтобы ослабить бота, а именно: выбрать самый редкий цвет из тех, что есть на руке, для того, чтобы к следующему ходу карт, которыми можно было бы походить, было меньше.

Поочередно идет проверка всех карт на руке на цвет, самый редковстречаемый будет выбран.

8. **void handleSetStarted(int gameNumber)** - перегруженный метод, который предупреждает о начале новой игры.

2.2.2 SmartBot

Большинство методов совпадает с классом **UnoBot**.

1. **string name() const** - возвращает имя игрока.

2. **void receiveCards(const vector<const Card*> cards)** - метод который отвечает за получение карт на руку.

В данном методе класс игрока получает карты в руку во время игры, копирует их в поле класса **Hand**, в то же время считая количество цветов и карточных действий в поле класса **clrs**. Для того, чтобы в новой партии у него на руках не оставались карты с предыдущей, исполь-

зуюм метод **clear()** к методу класса **Hand**. Но используется он в перегруженном методе **void handleSetStarted (int gameNumber)** который оповещает всех игроков о старте новой партии.

3. **const Card* playCard()** - этот метод позволяет ходить картой с руки.

Карты перебираются по порядку пока не найдется подходящая карта. Если первой картой удовлетворяющей условию стала "Дикая карта" , тогда перебор пройдет дальше в поиске другой карты. Если карт подходящих по цвету и значению не нашлось, играть будет "Дикая карта" , если нет ни одной подходящей карты, бот возьмет карту из колоды.

4. **bool isMatchingCard(const Card* card) const** - функция проверяет подходят ли карта на руке для хода.

Функция проверяет поочередно карты на руке по цвету и значению. Если карта подходит по цвету или по значению, то функция выдаст **true**, а также результат будет **true**, если набранная карта является картой типа "Дикая карта" , в остальных случаях функция выдаст **false**.

5. **void playMatchingCard(const Card* card)** - этот метод нужен для того, чтобы сыграть картой.

Если результат функции **bool isMatchingCard(const Card* card) const true**, значит карта с руки подходящая для игры в этом раунде, а значит она будет сыграна. Функция также позволяет играть только что набранными картами из колоды.

6. **bool drawAdditionalCard(const Card* additionalCard)** - Функция, определяющая подходит ли набранная карта из колоды для того, чтобы сыграть ее в этом же раунде.

Набранная карта сравнивается по цвету и по значению с последней картой на столе, а также принадлежит ли набранная карта к типу "Дикая карта" .Если карта соответствует по цвету, или по значению, или принадлежит к типу "Дикая карта" , Функция выводит **true**, если же нет **false**.

7. **CardColor changeColor()** - функция, которая позволяет сделать самый рациональный ход. Если выбрать цвет, которого больше всего на руках, шанс, что к следующему ходу этот цвет останется значительно выше, того, что цвет поменяется, следовательно шанс, что в следующий ход бот сможет походить будет гораздо выше.

Бот подсчитывает, какого из всех цветов больше всего на руках. Самый частовстречаемый цвет на руках будет выбран как цвет, на который поменяется предыдущий.

8. **void handleSetStarted(int gameNumber)** - перегруженный метод, который предупреждает о начале новой игры.

9. **void rememberDiscardedCards(const vector<const Card*> cards)** - функция запоминающая карты, отправляемые в сброс.

Функция запоминает цвет и значение каждой сыгранной карты на столе. Идея в том, что палаясь на то, какие карты уже вышли можно понять какие карты еще в колоде или на руках у соперника. Зная, что каждое значения кроме нуля встречается всего 8 раз можно считать, сколько карт каждого значения еще будет сыграно, а значит можно понять, что карт по значению,

которых осталось в колоде меньше нужно играть как можно скорее, потому что шанс, что последняя карта по значению нам подойдет будет меньше. Такая же логика повторяется и с цветом. Также учитывается то, что по окончании колоды сброс обнуляется до последней карты.

10. **const Card* findCardInHandWithSameValue(const Card* card) const** - функция которая нужна как раз для того, чтобы сравнивать значения карт на руках с теми, что уже попали в сброс.

Она считает, как много карт еще в колоде или у оппонентов, которые могут подойти боту по значению и определяет, каких совпадений меньше всего. Исходя из этого решается какой картой желательно ходить.

11. **const Card* findCardInHandWithSameColor(const Card* card) const** - функция которая сравнивает цвет карт на руках с теми, что уже попали в сброс.

Она считает, как много карт еще в колоде или у оппонентов, которые могут подойти боту по цвету и определяет, каких совпадений меньше всего. Исходя из этого решается какой картой желательно ходить.

3 Результаты

С помощью встроенной функции **statistic Test()** получилось узнать с каким средним процентом выигрывает умный бот - **SmartBot** у неумного бота - **UnoBot**: в 1000 играх 1 на 1 умный бот в среднем выигрывает при 65%.

Выводы

В ходе работы были реализованы два класса **SmartBot** и **UnoBot**, которые работают корректно и без ошибок. Оба класса унаследованы от абстрактного класса **UnoPlayer**. Все необходимые виртуальные функции перегружены. Все поставленные задачи были выполнены.

На работу было потрачено около 18 часов. Классы игроков были разработаны в среде разработки Visual Studio 2022 (v143). Также использовался стандарт языка C++ 14 и компилятор MSVC версии 11,35,32215.

Приложение А. Исходный код класса SmartBot

А.1. SmartBot.h

```
#pragma once
#include <vector>
#include "prod/src/game/uno_game.h"
using std::string;
using std::vector;

class SmartBot : public UnoPlayer
{
    string plname;
    int ctr_h;
    vector<const Card*> Hand;
    vector<unsigned int> clr;
    const Card* LastCard;
    CardColor selectedColor;
    vector<const Card*> discardPile; // Карты, уходящие в сброс
    vector<const Card*> cardsInDiscard; // Карты, запомненные в сбросе

public:
    SmartBot(const char* name);

    string name() const override;

    void receiveCards(const vector<const Card*>& cards);

    const Card* playCard() override;

    bool isMatchingCard(const Card* card) const;

    void playMatchingCard(const Card* card);

    bool drawAdditionalCard(const Card* additionalCard) override;

    CardColor changeColor() override;
```

```
void handleSetStarted(int gameNumber);

void rememberDiscardedCards(const vector<const Card*>& cards);

const Card* findCardInHandWithSameValue(const Card* card) const;

const Card* findCardInHandWithSameColor(const Card* card) const;
};
```

A.2. SmartBot.cpp

```
#include "SmartBot.h"

SmartBot::SmartBot(const char* name) : pname(name), ctr_h(0), LastCard(nullptr) {}

string SmartBot::name() const {
    return pname;
}

void SmartBot::receiveCards(const vector<const Card*>& cards) {
    const Card* topCard = game()->topCard();

    for (int i = 0; i < cards.size(); i++) {
        Hand.push_back(cards[i]);
        ctr_h++;

        switch (cards[i]->color) {
            case Red:
                clr[0]++;
                break;
            case Green:
                clr[1]++;
                break;
            case Blue:
                clr[2]++;
                break;
            case Yellow:
                clr[3]++;
                break;
        }

        switch (cards[i]->value) {
            case Wild:
                if (topCard != nullptr) {
                    clr[topCard->color]--; // Замена цвета последней сыгранной карты
                }
            }
        }
    }
}
```

```

    }

    clrs[4]++; // Увеличение количества карт Wild
    break;
case WildDraw4:
    if (topCard != nullptr) {
        clrs[topCard->color]--; // Замена цвета последней сыгранной карты
    }
    clrs[5]++; // Увеличение количества карт WildDraw4
    break;
}
}

if (topCard == nullptr) {
    selectedColor = changeColor(); // Установка желаемого цвета, если topCard равен nullptr
}
}

const Card* SmartBot::playCard() {
    CardColor currentColor = game()->currentColor();
    const Card* topCard = game()->topCard();

    if (topCard == nullptr) {
        LastCard = nullptr; // Устанавливаем LastCard в nullptr, если верхней карты нет
        return nullptr; // Если верхней карты нет, возвращаем nullptr
    }

    if (cardsInDiscard.size() >= 2)
    {
        for (const Card* discardCard : cardsInDiscard)
        {
            const Card* matchingCard = findCardInHandWithSameValue(discardCard);
            if (matchingCard != nullptr)
            {
                Hand.erase(std::find(Hand.begin(), Hand.end(), matchingCard));
                ctr_h--;
                LastCard = matchingCard;
            }
        }
    }
}

```

```

        return matchingCard;
    }
}

for (const Card* discardCard : cardsInDiscard)
{
    const Card* matchingCard = findCardInHandWithSameColor(discardCard);
    if (matchingCard != nullptr)
    {
        Hand.erase(std::find(Hand.begin(), Hand.end(), matchingCard));
        ctr_h--;
        LastCard = matchingCard;
        return matchingCard;
    }
}

}

for (int i = 0; i < ctr_h; i++) {
    const Card* card = Hand[i];

    if (isMatchingCard(card)) {
        if (card->value == 14 && clrns[currentColor] != 0) {
            continue;
        }
        else {
            Hand.erase(Hand.begin() + i);
            ctr_h--;

            switch (card->color) {
            case Red:
                clrns[0]--;
                break;
            case Green:
                clrns[1]--;
                break;
            case Blue:

```

```

        clr[2]--;
        break;
    case Yellow:
        clr[3]--;
        break;
    }

    switch (card->value) {
    case 13:
        clr[4]--;
        clr[game()->currentColor()]++;
        break;
    case 14:
        clr[5]--;
        clr[game()->currentColor()]++;
        break;
    }

    LastCard = card;
    return card;
}

}

// Если не найдено подходящей карты, взять карту из колоды

if (drawAdditionalCard(topCard) == true)
{
    LastCard = nullptr;
    return nullptr;
}

LastCard = nullptr; // Устанавливаем LastCard в nullptr, если не найдено подходящей карты
return nullptr; // Если не найдено подходящей карты, возвращаем nullptr // Если нет подходящей
}

bool SmartBot::isMatchingCard(const Card* card) const {

```

```

    if (card->color == game()->currentColor() ||
        card->value == Wild ||
        card->value == WildDraw4 ||
        card->value == game()->topCard()->value) {
        return true;
    }
    // Проверяем, совпадает ли цвет или ранг карты с последней сброшенной картой
    return false;
}

```

```

void SmartBot::playMatchingCard(const Card* card) {
    for (int i = 0; i < ctr_h; i++) {
    if (Hand[i] == card) {
    switch (card->color) {
    case Red:
    clr[0]--;
    break;
    case Green:
    clr[1]--;
    break;
    case Blue:
    clr[2]--;
    break;
    case Yellow:
    clr[3]--;
    break;
    }
}

```

```

switch (card->value) {
case Wild:
clr[4]--;
clr[game()->currentColor()]++;
break;
case WildDraw4:
clr[5]--;
clr[game()->currentColor()]++;
}

```

```

break;
}

Hand.erase(Hand.begin() + i);
ctr_h--;
LastCard = card;
break;
}
}
}

bool SmartBot::drawAdditionalCard(const Card* additionalCard) {
    if (isMatchingCard(additionalCard)) {
        // Если полученная карта подходит, сразу же сыгрываем ее
        playMatchingCard(additionalCard);
        return true;
    }
    else {
        // Если полученная карта не подходит, добавляем ее в руку
        Hand.push_back(additionalCard);
        ctr_h++;

        return false;
    }
}

CardColor SmartBot::changeColor() {
    vector<int> colorCount(4, 0); // Индексы 0-3 соответствуют Red, Green, Blue, Yellow

    for (const auto& card : Hand) {
        if (card->color == Red)
            colorCount[Red]++;
        else if (card->color == Green)
            colorCount[Green]++;
        else if (card->color == Blue)
            colorCount[Blue]++;
    }
}

```

```

        else if (card->color == Yellow)
            colorCount[Yellow]++;
    }

    CardColor leastCommonColor; // Начальное значение - отсутствие цвета
    int leastCount = INT_MIN; // Начальное значение - максимальное целое число

    for (int i = Red; i <= Yellow; i++) {
        if (colorCount[i] > leastCount) {
            leastCount = colorCount[i];
            leastCommonColor = static_cast<CardColor>(i);
        }
    }

    return leastCommonColor;
}

void SmartBot::handleSetStarted(int gameNumber) {
    clr.clear();
    for (int i = 0; i < 6; i++) {
        clr.push_back(0);
    }
    Hand.clear();
    ctr_h = 0;
    selectedColor = changeColor();
}

void SmartBot::rememberDiscardedCards(const vector<const Card*>& cards)
{
    // Запомните карты, отправляемые в сброс
    for (const Card* card : cards)
    {
        cardsInDiscard.push_back(card);
    }
}

```

```
}
```

```
const Card* SmartBot::findCardInHandWithSameValue(const Card* card) const
{
    // Находим карту с тем же значением в руке
    for (const Card* handCard : Hand)
    {
        if (handCard->value == card->value)
        {
            return handCard;
        }
    }
    return nullptr;
}
```

```
const Card* SmartBot::findCardInHandWithSameColor(const Card* card) const
{
    // Находим карту с тем же цветом в руке
    for (const Card* handCard : Hand)
    {
        if (handCard->color == card->color)
        {
            return handCard;
        }
    }
    return nullptr;
}
```

Приложение Б. Исходный код класса UnoBot

Б.1. UnoBot.h

```
#pragma once
//#include <iostream>
#include <string>
#include <vector>
#include "prod/src/game/uno_game.h"
//#include "prod/src/utlis/stats.h"
//#include "prod/src/utlis/logger.h"
//using namespace std;
using std::string;
using std::vector;
//using std::cout;
//using std::endl;

class UnoBot : public UnoPlayer
{
    string plname;
    int ctr_h;
    vector<const Card*> Hand;
    vector<unsigned int> clrs;
    const Card* LastCard;
    CardColor selectedColor;

public:

    UnoBot(const char* name);

    string name() const override;

    void receiveCards(const vector<const Card*>& cards);/// @brief Игрок получает на руки карты.

    const Card* playCard() override;/// @brief Игрок возвращает карту, которую он сыграет (полож
```

```

bool isMatchingCard(const Card* card) const;

void playMatchingCard(const Card* card);

bool drawAdditionalCard(const Card* additionalCard) override; /// @brief Если у игрока нет п

CardColor changeColor() override;/// @brief Если игрок положил "Закажи цвет" или "Возьми чет

void handleSetStarted(int gameNumber);

};

```

Б.2. UnoBot.cpp

```
#include "UnoBot.h"

UnoBot::UnoBot(const char* name) {
    pname = name;
    ctr_h = 0; // Инициализация счетчика
    clrs.resize(6, 0); // Инициализация размера и заполнение нулями
    //Функция resize вектора используется для изменения размера вектора. Она принимает два аргумента
    //размер вектора и значение, которым заполняются новые элементы, если новый размер больше текуще
}

string UnoBot::name() const {
    return pname;
}

void UnoBot::handleSetStarted(int gameNumber) {
    clrs.clear();
    for (int i = 0; i < 6; i++) {
        clrs.push_back(0);
    }
    Hand.clear();
    ctr_h = 0;
    selectedColor = changeColor();
}

void UnoBot::receiveCards(const vector<const Card*>& cards) {
    const Card* topCard = game()->topCard();

    for (int i = 0; i < cards.size(); i++) {
        Hand.push_back(cards[i]);
        ctr_h++;

        switch (cards[i]->color) {
            case Red:
                clrs[0]++;
            default:
                break;
        }
    }
}
```

```

break;
case Green:
    clr[1]++;
break;
case Blue:
    clr[2]++;
break;
case Yellow:
    clr[3]++;
break;
}

switch (cards[i]->value) {
case Wild:
    if (topCard != nullptr) {
        clr[topCard->color]--; // Замена цвета последней сыгранной карты
    }
    clr[4]++; // Увеличение количества карт Wild
break;
case WildDraw4:
    if (topCard != nullptr) {
        clr[topCard->color]--; // Замена цвета последней сыгранной карты
    }
    clr[5]++; // Увеличение количества карт WildDraw4
break;
}

if (topCard == nullptr) {
    selectedColor = changeColor(); // Установка желаемого цвета, если topCard равен nullptr
}
}

const Card* UnoBot::playCard() {
    CardColor currentColor = game()->currentColor();
    const Card* topCard = game()->topCard();

```

```

if (topCard == nullptr) {
LastCard = nullptr; // Устанавливаем LastCard в nullptr, если верхней карты нет
return nullptr; // Если верхней карты нет, возвращаем nullptr
}

for (int i = 0; i < ctr_h; i++) {
const Card* card = Hand[i];

if (isMatchingCard(card)) {
if (card->value == 14 && clr[currentColor] != 0) {
continue;
}
else {
Hand.erase(Hand.begin() + i);
ctr_h--;

switch (card->color) {
case Red:
clr[0]--;
break;
case Green:
clr[1]--;
break;
case Blue:
clr[2]--;
break;
case Yellow:
clr[3]--;
break;
}

switch (card->value) {
case 13:
clr[4]--;
clr[game()->currentColor()]++;

```

```

break;
case 14:
    clr[5]--;
    clr[game()->currentColor()]++;
break;
}

```

```

LastCard = card;
return card;
}
}
}

```

```

LastCard = nullptr; // Устанавливаем LastCard в nullptr, если не найдено подходящей карты
return nullptr; // Если не найдено подходящей карты, возвращаем nullptr
}

```

```

bool UnoBot::isMatchingCard(const Card* card) const { //функция проверяет подходит ли набранная к
if (card->color == game()->currentColor() ||
card->value == Wild ||
card->value == WildDraw4 ||
card->value == game()->topCard()->value) {
return true;
}
return false;
}

```

```

void UnoBot::playMatchingCard(const Card* card) { //если да мы играем этой картой сразу
for (int i = 0; i < ctr_h; i++) {
if (Hand[i] == card) {
switch (card->color) {
case Red:
    clr[0]--;
break;
case Green:

```

```

    clr[1]--;
    break;
    case Blue:
    clr[2]--;
    break;
    case Yellow:
    clr[3]--;
    break;
}

switch (card->value) {
    case Wild:
    clr[4]--;
    clr[game()->currentColor()]++;
    break;
    case WildDraw4:
    clr[5]--;
    clr[game()->currentColor()]++;
    break;
}

Hand.erase(Hand.begin() + i);
ctr_h--;
LastCard = card;
break;
}
}
}

bool UnoBot::drawAdditionalCard(const Card* additionalCard) {
    if (isMatchingCard(additionalCard)) {
        // Если полученная карта подходит, сразу же сыгрываем ее
        playMatchingCard(additionalCard);
        return true;
    }
    else {

```

```
// Если полученная карта не подходит, добавляем ее в руку
```

```
Hand.push_back(additionalCard);
```

```
ctr_h++;
```

```
return false;
```

```
}
```

```
}
```

```
CardColor UnoBot::changeColor() {
```

```
vector<int> colorCount(4, 0); // Индексы 0-3 соответствуют Red, Green, Blue, Yellow
```

```
for (const auto& card : Hand) {
```

```
if (card->color == Red)
```

```
colorCount[Red]++;
```

```
else if (card->color == Green)
```

```
colorCount[Green]++;
```

```
else if (card->color == Blue)
```

```
colorCount[Blue]++;
```

```
else if (card->color == Yellow)
```

```
colorCount[Yellow]++;
```

```
}
```

```
vector<CardColor> leastCommonColors; // Список цветов с наименьшим количеством карт
```

```
int leastCount = INT_MAX; // Начальное значение - максимальное целое число
```

```
for (int i = Red; i <= Yellow; i++) {
```

```
if (colorCount[i] < leastCount && colorCount[i] > 0) { // Исключаем цвета с нулевым количеством
```

```
leastCount = colorCount[i];
```

```
leastCommonColors.clear();
```

```
leastCommonColors.push_back(static_cast<CardColor>(i));
```

```
}
```

```
else if (colorCount[i] == leastCount && colorCount[i] > 0) { // Добавляем цвета с равным наимень
```

```
leastCommonColors.push_back(static_cast<CardColor>(i));
```

```
}
```

```
}
```

```
if (leastCommonColors.empty()) {
```

```
// Если нет цветов с ненулевым количеством карт, вернуть случайный цвет
return static_cast<CardColor>(rand() % 4);
}
else {
// Из списка наименьших цветов выбрать случайный
int randomIndex = rand() % leastCommonColors.size();
return leastCommonColors[randomIndex];
}
}
```